A Research Fundamental Project Proposal on

BUILDING A GPT-124M MODEL FROM SCRATCH

Submitted in partial fulfillment of the requirements for the Practical Marks of

RESEARCH FUNDAMENTALS

By

BIKRAM SHARMA

PRAMESH LAMICHHANE

SWIKAR POUDEL

Supervisor

Bidur Devkota, PhD

Department of Research and Development

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

Lamachaur, Kaski, Nepal

(JULY, 2026)

APPROVAL CERTIFICATE

This project entitled "**BUILDING A GPT-124M MODEL FROM SCRATCH**", prepared and submitted by "**Bikram Sharma, Pramesh Lamichhane, Swikar Poudel**" under the supervision of "**Bidur Devkota**" in partial fulfillment of the requirements for the **Practical Marks for Research Fundamentals** has been examined and is recommended for approval and acceptance.

Date of Evaluation: July 10, 2026

……

Bidur Devkota, PhD

(Project Supervisor)

……

Ishwor Koirala, PhD

Project Head

Research Management Committee

Gandaki College of Engineering and Science

ABSTRACT

Large language models have become part of everyday computing, yet most students only see their output, never the mechanism behind it. This proposal describes a minor project that builds a GPT-124M-style language model from scratch, using the public repository GPT-From-scratch as a starting point rather than a finished product. The work covers byte-pair tokenization, learned token and positional embeddings, causal multi-head self-attention, transformer blocks with residual connections and layer normalization, cross-entropy training with validation tracking, checkpointing, and controlled decoding through temperature and top-k sampling.

Beyond reproducing the architecture, the project adds the engineering discipline the original repository lacks: a modular code structure, reproducible configuration, automated unit tests, a documented evaluation protocol with loss and perplexity curves, and a complete set of design artifacts, including an architecture diagram, use case diagram, ER diagram, sequence diagrams, class diagram, Gantt chart, and risk matrix. The expected outcome is a small, fully working GPT-style model trainable end-to-end on a chosen corpus, evaluated quantitatively, and capable of generating short text passages, while remaining honest about its scale and limitations relative to commercial large language models.

Keywords: GPT, Transformer, PyTorch, Language Model, Tokenization, Self-Attention, Text Generation, Deep Learning, Software Engineering

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

Abbreviations	Meaning
AI	Artificial Intelligence
BPE	Byte Pair Encoding
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DL	Deep Learning
ER	Entity Relationship
FFN	Feed-Forward Network
GPU	Graphics Processing Unit
GPT	Generative Pre-trained Transformer
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
RAM	Random Access Memory
SSD (in diagrams)	System Sequence Diagram
UML	Unified Modeling Language

Table 1: List of Abbreviations

Chapter 1

INTRODUCTION

1.1 Background

Language models built on the Transformer architecture have become one of the defining technologies of the current decade. The Transformer replaced recurrent connections with self-attention, allowing a model to weigh every token in a sequence against every other token in parallel, which made large-scale training practical for the first time (Vaswani et al., 2017). GPT-style models took this idea and specialized it for autoregressive generation: at every position, the model is trained to predict the next token using only the tokens that came before it (Radford et al., 2019). That single, almost embarrassingly simple training objective, when scaled up with enough data and parameters, is what underlies modern assistants, coding tools, and chatbots.

Most students only ever see the output side of this technology — a chat window or an API response. The mechanism that produces that output, the matrix multiplications, masking, and gradient updates happening underneath, stays invisible. This project deliberately works against that trend. Rather than calling a pretrained model, it builds one: a byte-pair tokenizer turns raw text into integers, an embedding table turns integers into vectors, a stack of transformer blocks mixes information across positions using causal self-attention, and a final linear layer turns hidden states back into a probability distribution over the vocabulary.

The starting point for the implementation is the public GitHub repository GPT-From-scratch (https://github.com/beeks-code/GPT-From-scratch), which already contains a working GPT-124M configuration (config.py), a transformer implementation (model/transformer.py and model/gpt.py), a training notebook, a series of tutorial notebooks covering tokenization and attention, and a small sample corpus (the-verdict.txt). This proposal does not treat that repository as a finished product; it treats it as a prototype that needs the structure, documentation, and verification that a real software project requires before it can be called complete.

1.2 Problem Statement

There is no shortage of people who can prompt a language model well. There is a much smaller group who can explain, line by line, how a prompt turns into a generated

sentence. Tutorials on the subject tend to teach tokenization, attention, training, and decoding as separate, disconnected exercises, which leaves learners with fragments rather than a working system. At the same time, a repository of research code, however correct, is not the same thing as an engineered project: it usually lacks reproducible configuration, automated tests, a documented evaluation methodology, and the design diagrams that let someone other than the original author understand the system quickly.

The problem this project addresses is therefore twofold: first, to implement a complete, internally consistent GPT-style language-model pipeline — tokenizer, dataset, model, training loop, evaluation, and generation — from first principles; and second, to wrap that implementation in the software-engineering practice (modular structure, configuration management, testing, documentation, and design diagrams) needed to make it a credible, reproducible minor project rather than a personal experiment.

Proposed Solution: The proposed response is to treat the GPT-From-scratch repository as a starting prototype rather than a finished artifact, and to rebuild it in three layers: a verified model layer, in which the tokenizer, embeddings, attention mechanism, transformer blocks, and output head are each covered by shape and correctness tests; a training and evaluation layer, in which the training loop is documented, validation loss and perplexity are tracked at fixed intervals, and checkpoints are saved for reproducibility; and a project layer, comprising design diagrams, a feasibility analysis, a risk register, and an explicit ethical-use statement, so the finished work can be evaluated as a complete engineering artifact rather than a single training script.

1.3 Objectives

- To study the Transformer and GPT architecture in depth, from the original research papers through to modern educational implementations such as nanoGPT.
- To implement a GPT-style decoder-only model in PyTorch, covering token and positional embeddings, masked multi-head self-attention, position-wise feed-forward networks, layer normalization, residual connections, and an output projection head.
- To build a tokenized dataset pipeline using GPT-2 byte-pair encoding (via tiktoken) and fixed-length context windows with configurable stride.

- To train and validate the model on a selected text corpus, recording training loss, validation loss, approximate perplexity, and sample generations at regular checkpoints.
- To implement and compare decoding strategies — greedy decoding, temperature sampling, and top-k sampling — for controllable text generation.
- To refactor the existing repository into a clean, modular, and reproducible codebase with automated unit tests, configuration logging, and checkpoint management.
- To produce a complete set of software design artifacts (block diagram, use case diagram, ER diagram, sequence diagrams, class diagram, Gantt chart, risk matrix) appropriate for academic evaluation.

1.4 Scope and Limitation

The project is scoped as an educational, single-GPU-or-CPU-feasible implementation. It is not intended to compete with, or approximate the performance of, production language models such as GPT-3, GPT-4, LLaMA, or similar systems, which are trained on internet-scale corpora using thousands of accelerators. The proposed system uses a GPT-124M-style configuration (50,257-token vocabulary, 768-dimensional embeddings, 12 attention heads, 12 transformer layers) but trains it on a comparatively small, manageable corpus, so the realistic outcome is a model that demonstrably learns local structure in text — spelling, common phrases, simple grammar — rather than one that produces fluent, knowledge-rich prose. This boundary is treated as a feature of the project, not a weakness: the goal is transparency and correctness of implementation, not state-of-the-art output quality.

1.5 Application and Implications

Beyond its immediate academic purpose, the project has several practical extensions. The same pipeline can be repointed at a different corpus — for example, Nepali-language text — to study how tokenization and training behave on a low-resource language. The modular structure makes it straightforward to swap in a different tokenizer, vary model depth or width to study scaling behavior empirically, or add a lightweight web front end for interactive prompting. Because every component is visible and testable, the project can also serve as a teaching artifact for future research-

fundamentals cohorts, and as a base for later coursework touching on model evaluation, bias, or hallucination in generative systems.

Chapter 2

LITERATURE REVIEW

The technical foundation for this project draws on a small set of papers that, together, explain why the Transformer architecture works and how it scales, plus one well-known educational reference implementation. The table below summarizes each source and how it is used in this proposal.

Source	Main Contribution	Relevance to this project
Vaswani et al. (2017), "Attention Is All You Need"	Introduced the Transformer architecture and scaled dot-product self-attention, replacing recurrence with parallelizable attention.	Provides the mathematical and architectural basis for the transformer block, multi-head attention, and positional encoding used in this project.
Radford et al. (2019), GPT-2 paper	Showed that a decoder-only Transformer trained purely on next-token prediction at scale exhibits broad, unsupervised multitask behavior.	Justifies the choice of an autoregressive, decoder-only architecture and the GPT-2 byte-pair tokenizer.
Sennrich, Haddow & Birch (2016)	Introduced byte-pair encoding (BPE) for open-vocabulary neural machine translation.	Underpins the subword tokenization strategy (via tiktoken) used to convert text into model input.

Source	Main Contribution	Relevance to this project
Kaplan et al. (2020), "Scaling Laws for Neural Language Models"	Empirically characterized how loss scales with model size, dataset size, and compute.	Explains, quantitatively, why a 124M-parameter model trained on a small corpus cannot be expected to match production LLM fluency, and motivates realistic scope-setting.
Hoffmann et al. (2022), "Training Compute-Optimal Large Language Models" (Chinchilla)	Showed that many large models of the time were under-trained relative to their parameter count, and proposed compute-optimal data-to-parameter ratios.	Informs the project's decision to favor a smaller, fully-trained configuration over an under-trained large one within the available compute budget.
Wolf et al. (2020), Hugging Face Transformers	Described a widely used, production-grade transformer library and its design choices.	Used as a comparison point for API design, configuration handling, and the gap between research code and production tooling.
Karpathy (2023), nanoGPT	A widely cited minimal, readable GPT training codebase built for education.	Serves as a style and structure reference for a clean training loop, and as a sanity check on expected loss curves for small-scale training.

Source	Main Contribution	Relevance to this project
beeks-code (2026), GPT-From-scratch	The base repository for this project, containing GPT-124M configuration, model code, tutorials, and a sample corpus.	Forms the literal starting codebase that this proposal extends into a complete academic project.

Table 2: Summary of Literature Reviewed

2.1 Synthesis and Positioning

Two consistent themes emerge from this review. First, the core mechanism behind GPT-style models is not mysterious: self-attention, applied causally and stacked in depth, is sufficient to model long-range dependencies in text, and the engineering challenge lies in implementing it correctly and efficiently rather than in any single conceptual leap. Second, the scaling-law literature (Kaplan et al., 2020; Hoffmann et al., 2022) makes it clear that quality is strongly compute- and data-dependent, which is precisely why this proposal treats "correct, well-tested, well-documented small model" as success, rather than chasing fluency that the available hardware cannot realistically deliver. The project's contribution, in that context, is not a new architecture but a disciplined, fully verified, and clearly documented implementation — something the literature reviewed here describes conceptually but rarely walks through end-to-end at a scale a student can reproduce on a laptop or a single cloud GPU.

Chapter 3

TOOLS AND METHODOLOGY

3.1 Required Tools

Tool/Library	Purpose
Python 3.10+	Primary implementation language for the model, dataset pipeline, and training scripts.
PyTorch	Tensor operations, autograd, neural network modules, optimizer, and GPU acceleration.
tiktoken	GPT-2 byte-pair tokenizer for encoding text to token ids and decoding ids back to text.
Jupyter Notebook	Interactive experimentation, tutorial walkthroughs, and training/evaluation logs.
Git and GitHub	Version control and collaboration. Base repository: https://github.com/beeks-code/GPT-From-scratch
pytest	Automated unit and integration testing for tensor shapes, masking, loss, and generation.
NumPy / Pandas / Matplotlib	Metric aggregation, tabulation, and plotting of loss/perplexity curves.
VS Code	Code editing, debugging, and Git integration during development.
GPU runtime (CUDA) / CPU fallback	Model training; GPU preferred for full runs, CPU acceptable for unit tests and small-scale validation.
Microsoft Word / PowerPoint	Preparation of the proposal, final report, and defence presentation.

Table 3: Tools and Technology Stack

3.2 Existing Repository Study

Before proposing changes, the team studied the GPT-From-scratch repository directly. Its README describes the goal as “building GPT-124M from scratch.” The configuration file defines a vocabulary size of 50,257 tokens (matching GPT-2’s BPE vocabulary), a context length of 1,024 tokens, an embedding dimension of 768, 12

attention heads, 12 transformer layers, a dropout rate of 0.1, and no bias terms in the QKV projections. The training and tutorial notebooks note a total parameter count of roughly 162 million once the token embedding matrix, positional embedding matrix, and the (separately weighted) output projection are all counted — a figure that is plausible for this configuration and is consistent with how the original GPT-2-small variant is sometimes reported in tutorials that do not tie input and output embedding weights together.

Repository Item	Role in the Project
config.py	Defines GPT_CONFIG_124M: vocabulary size, context length, embedding dimension, number of heads/layers, dropout.
model/gpt.py	Assembles GPT_124: token + positional embeddings, the transformer stack, final layer norm, output head.
model/transformer.py	Implements LayerNormalization, GELU activation, FeedForward, MultiHeadAttention, and the TransformerBlock.
model/train.ipynb	Contains the dataset class, train/validation split, loss computation, and the training loop.
Tutorial/*.ipynb	Step-by-step explanations of tokenization, BPE, dataset construction, positional encoding, attention, loss, and decoding.
the-verdict.txt	A short public-domain text corpus used for early-stage training and unit testing.

Table 4: Existing Repository Structure

3.3 Development Methodology

The team will follow a feature-driven, iterative development methodology. Rather than attempting the entire pipeline in one pass, the system is decomposed into independently understandable features — tokenizer, dataset, transformer block, training loop, evaluation, generation, and documentation — each of which is designed, implemented, unit-tested, and only then integrated with the rest. This approach suits a small team

working under a fixed academic timeline because it produces working, demonstrable increments every week rather than a single large deliverable at the end, and it isolates debugging: a failing test points to one feature, not the whole system.

3.3.1 Feature List

Feature	Description
Tokenizer pipeline	Encode raw text into GPT-2 BPE token ids; decode generated ids back into readable text.
Dataset pipeline	Build fixed-length, overlapping input-target windows for next-token prediction, with a stride parameter.
Transformer block	Implement masked multi-head self-attention plus a position-wise feed-forward network, wired with residual connections and layer normalization.
GPT model	Combine embeddings, the transformer stack, a final normalization layer, and the output projection head.
Training loop	Run forward passes, compute cross-entropy loss, backpropagate, and update weights with AdamW.
Evaluation	Track validation loss, approximate perplexity, and qualitative generated samples across checkpoints.
Generation	Implement greedy, temperature-based, and top-k decoding for controllable text generation.
Testing and documentation	Unit tests for every feature above, plus diagrams, README, and proposal/report material.

Table 5: System Feature List

3.4 System Analysis and Design

The diagrams below were produced specifically for this proposal to describe the proposed system's structure and behavior at different levels of abstraction — a static architecture view, a use case view of who interacts with the system and how, a data view of what is recorded for reproducibility, and two dynamic sequence views of the system's two most important runtime behaviors: training and generation.

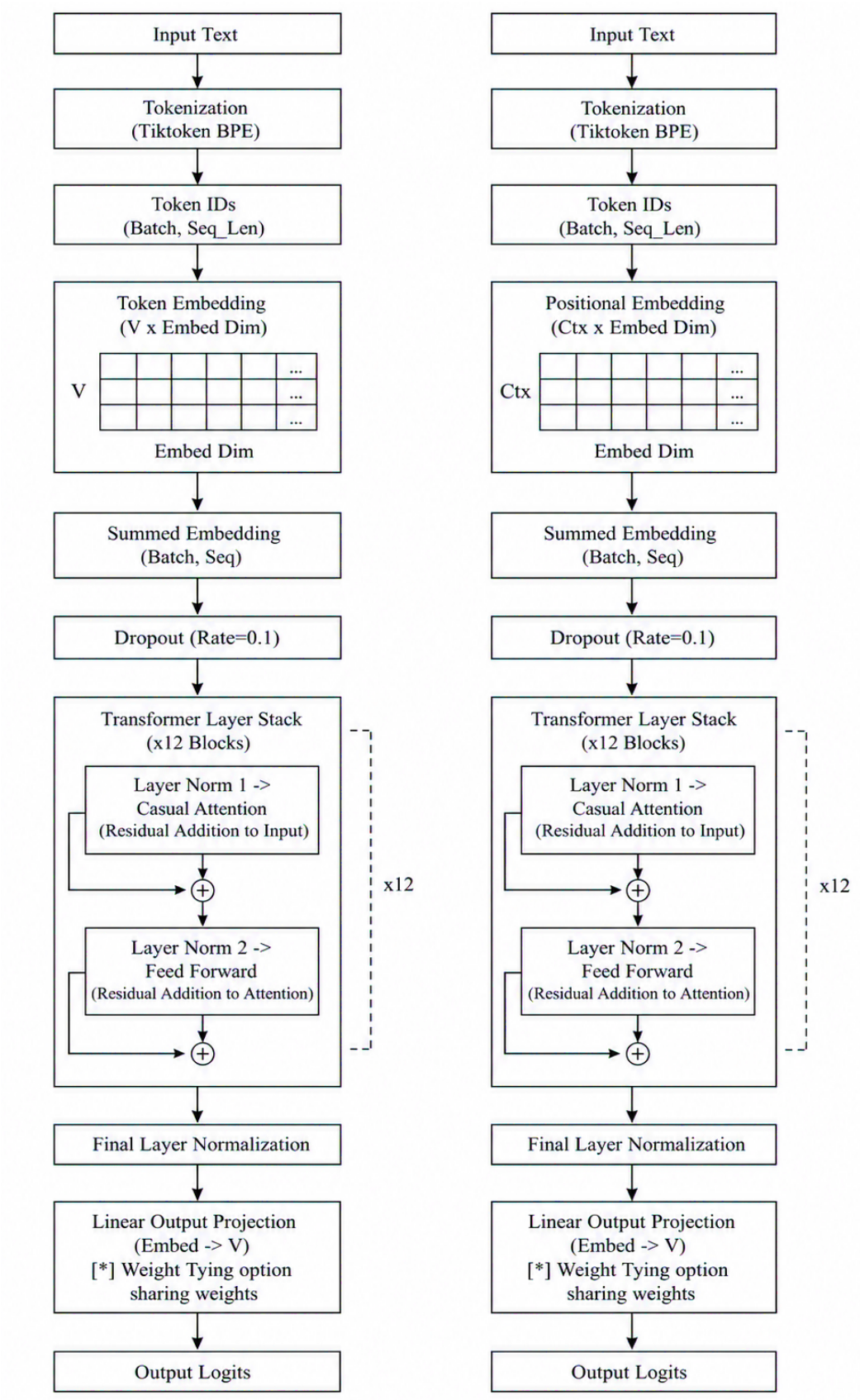

Figure 1: Proposed System Architecture

Figure 1 traces the path of data through the system: raw text is tokenized, organized into context windows, batched by a data loader, embedded, passed through a stack of transformer blocks, and projected to logits. The lower half of the diagram shows the training loop consuming those logits to compute loss and update weights (dashed orange path), and the decoding stage consuming the trained model's logits to produce generated text (solid path on the right).

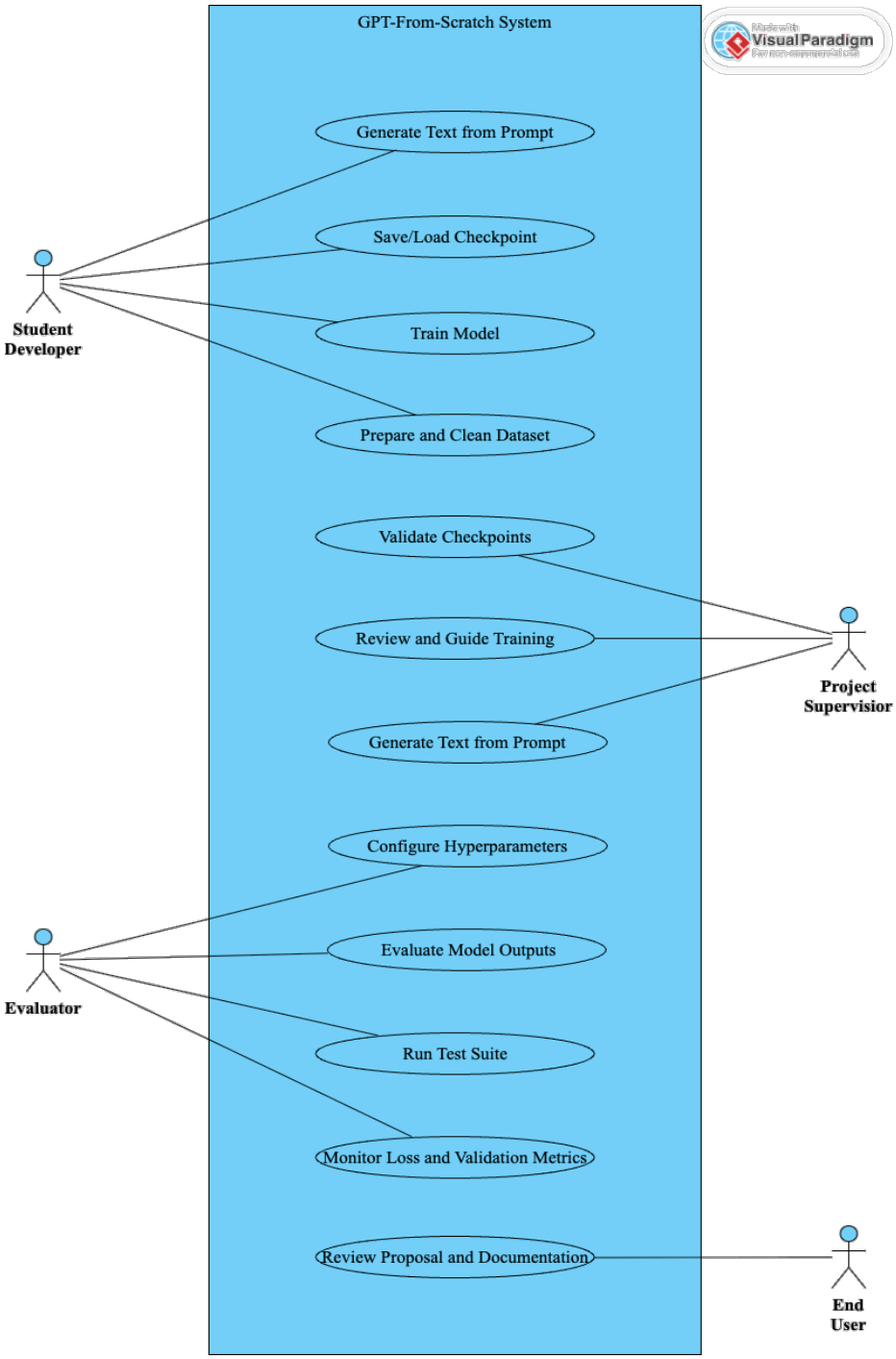

Figure 2: Use Case Diagram

Figure 2 identifies four actors — the student developer, the project supervisor, the evaluator/panel, and an end user — and the nine principal use cases they interact with, ranging from dataset preparation and model training to checkpoint management, testing, generation, and documentation review. Separating these actors makes explicit who is responsible for which part of the system during development versus evaluation.

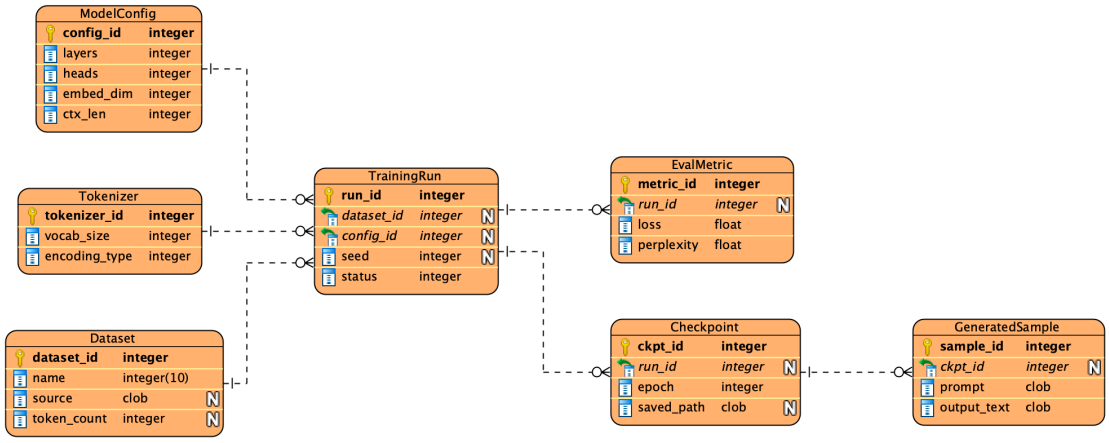

Figure 3: Entity Relationship Diagram

Because the project emphasizes reproducibility, Figure 3 defines a lightweight experiment-tracking schema: a Dataset and a Tokenizer feed into a ModelConfig, which together define a TrainingRun. Each run produces one or more Checkpoints, EvaluationMetrics, and, downstream of a checkpoint, GeneratedSamples. This structure means any reported result can be traced back to the exact configuration and data that produced it — a basic but often-skipped requirement of credible ML experimentation.

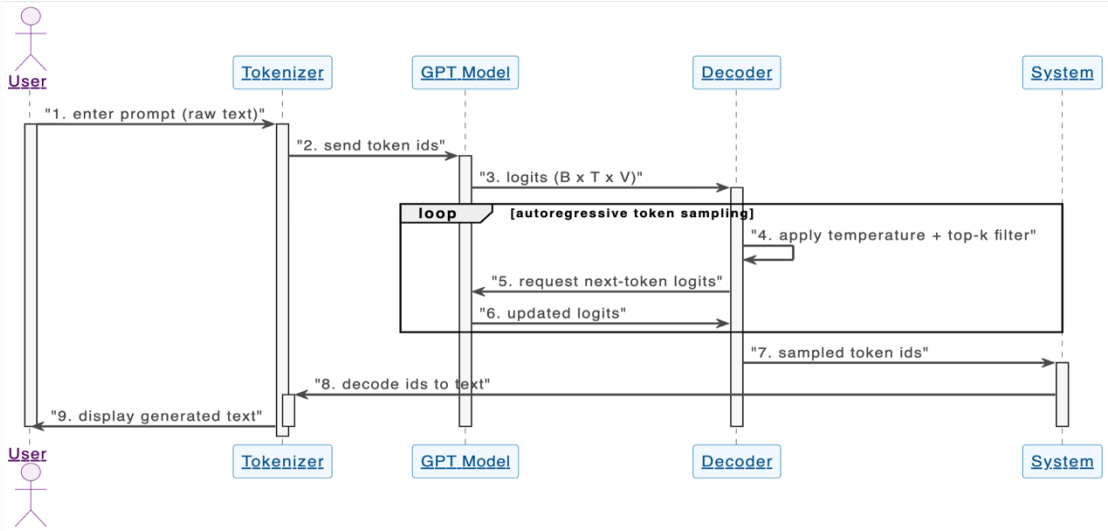

Figure 4: System Sequence Diagram - Text Generation

Figure 4 shows the runtime sequence for a single generation request: the user's prompt is tokenized, passed through the model to obtain logits, filtered by temperature and top-k before sampling, looped autoregressively until the desired length is reached, and finally decoded back into text for display.

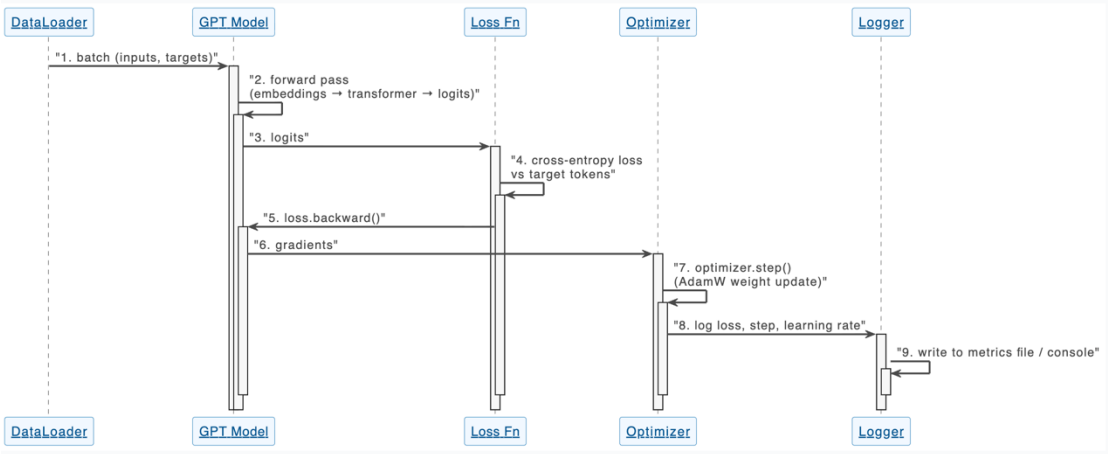

Figure 5: Sequence Diagram - One Training Iteration

Figure 5 shows the corresponding sequence for one training step: a batch is drawn from the data loader, a forward pass produces logits, cross-entropy loss is computed against the target tokens, gradients are backpropagated, AdamW updates the weights, and the resulting loss and learning rate are logged for later analysis.

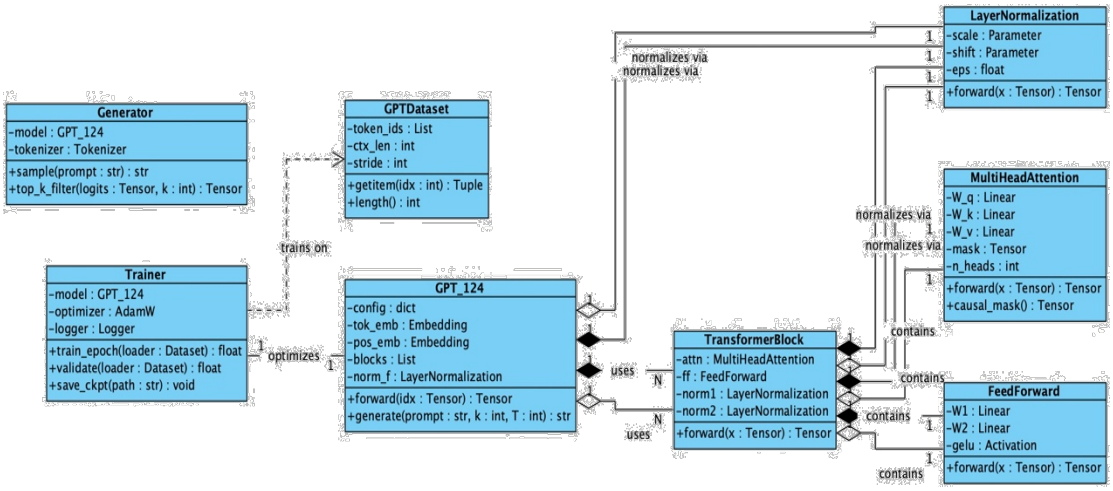

Figure 6: Class Diagram

Figure 6 maps the repository's existing code structure onto the classes the refactored project will expose: GPT_124 composes a stack of TransformerBlock instances, each of which contains a MultiHeadAttention and a FeedForward module and is normalized

by LayerNormalization; a GPTDataset class handles windowing; and Trainer and Generator classes wrap the training and inference workflows respectively, separating concerns that are currently mixed together inside notebook cells.

3.5 Algorithm and Approach

1. Collect and clean a text corpus, starting with the repository's existing sample text and keeping the pipeline open to a larger public-domain corpus if compute allows.
2. Encode text using the GPT-2 BPE tokenizer (tiktoken) and build sliding-window input-target token pairs with a configurable context length and stride.
3. Initialize the GPT-style model: token embeddings, positional embeddings, a stack of transformer blocks, a final layer normalization, and an output projection.
4. For each training batch, run a forward pass and compute cross-entropy loss between predicted logits and the target tokens shifted by one position.
5. Update weights with the AdamW optimizer, applying gradient clipping where needed, and log training and validation loss at fixed intervals.
6. Periodically generate text from a small set of fixed prompts using temperature and top-k sampling, to track qualitative learning progress alongside the quantitative loss curve.
7. Save configuration, model checkpoints, metrics, and generated samples after every evaluation interval so that any reported result is fully reproducible.

3.6 Proposed Improvements Beyond the Current Repository

The existing repository is a strong starting point but, like most personal research code, mixes concerns and lacks the scaffolding that makes a project auditable. The following improvements are proposed:

- Reorganize the codebase into a clean src/ structure that separates model, dataset, training, generation, and evaluation logic instead of leaving them inside notebook cells.
- Add YAML/JSON-based configuration logging so that every experiment run is fully reproducible from a single saved file.
- Add checkpoint saving and loading, including optimizer state, so training can be resumed or audited.

- Add a pytest-based suite covering tensor shapes, causal masking correctness, loss finiteness, and generation boundary conditions.
- Add a compact command-line interface (train.py, generate.py) so the system can be run without manually executing notebook cells.
- Add an evaluation notebook that plots loss and perplexity curves and tabulates generated samples across checkpoints.
- Prepare a single demo script that runs the full pipeline end-to-end for live defence demonstrations.

3.7 Feasibility Analysis

3.7.1 Technical Feasibility

The project is technically feasible: PyTorch provides stable, well-documented primitives for tensors, autograd, and neural network layers, and the base repository already demonstrates that the chosen architecture runs. The remaining work is incremental engineering — restructuring, testing, and instrumentation — rather than open research, which keeps technical risk low.

3.7.2 Economic Feasibility

Direct costs are minimal because every required tool (Python, PyTorch, tiktoken, Jupyter, Git/GitHub, pytest, VS Code) is free and open source. If a local GPU is unavailable, free-tier or low-cost cloud notebook sessions are sufficient for the scale of training planned here. The main practical costs are internet access, optional cloud compute credit, and printing/binding for the final report.

3.7.3 Operational Feasibility

The system can be demonstrated entirely through scripts and notebooks on a single laptop, with no server infrastructure or continuous deployment required. Evaluators can inspect the architecture, run a short training session live, load a pretrained checkpoint, and generate text from fixed prompts within a normal defence time slot.

3.8 Hardware and Software Requirements

S.No.	Requirement
1	Laptop or desktop with a multi-core CPU
2	Minimum 8 GB RAM (16 GB recommended for larger batch sizes)
3	CUDA-capable GPU preferred for full training runs; CPU acceptable for unit tests and small-scale validation
4	Stable internet connection for repository access, package installation, and optional cloud compute
5	Sufficient local or cloud storage for corpus files, checkpoints, notebooks, and generated outputs

Table 6: Hardware and Software Requirements

3.9 Participants and Team Roles

This is a three-member student team — Bikram Sharma, Pramesh Lamichhane, and Swikar Poudel — working under the supervision of Bidur Devkota, PhD. Rather than assigning fixed, siloed roles for the whole project, the team follows a shared-ownership model: every member is expected to be able to read, run, and explain every layer of the system, from tokenization through decoding. In practice, responsibility rotates by feature rather than by person; whoever picks up a feature from the list in Section 3.3.1 owns its implementation, its unit tests, and its portion of the documentation, while the other two members review the corresponding pull request before it is merged. This rotation is deliberate for a project of this size: no member should be confined to only the model code, only the training scripts, or only the written report, and all three should be able to answer questions on any part of the system at the final defense.

3.10 Data Collection and Analysis Methods

Data collection: The project does not involve human participants, and so does not require the ethical clearance procedures associated with human-subject research; the ethical considerations that do apply, mainly around corpus licensing and honest reporting, are addressed later in this chapter (Section 3.11). Data collection consists of selecting small, clearly licensed or public-domain text corpora rather than scraping new data: development and unit testing begin with the repository's existing sample corpus

(the-verdict.txt), and the team may add one further public-domain corpus of comparable size once the pipeline is verified, to confirm that training behaves consistently across sources.

Data analysis: Data analysis is primarily quantitative. Training and validation loss are logged at every evaluation interval, converted to approximate perplexity for interpretability, and plotted as loss-versus-step and loss-versus-epoch curves (Section 5.2) to check for convergence, instability, or overfitting. Generated text samples are examined qualitatively for coherence and corpus-specific vocabulary but are reported alongside the quantitative metrics rather than in place of them, so that results are not selectively presented.

Validation techniques: Validation of the trained system follows the structured test plan in Chapter 4 (Section 4.1), which verifies the tokenizer, the dataset windowing logic, the causal attention mask, the forward pass, the loss computation, checkpointing, and text generation independently before they are considered integrated.

3.11 Ethical and Responsible-Use Considerations

Even a small, educational language model raises questions worth addressing explicitly rather than leaving implicit. First, the project will train only on a small, clearly licensed or public-domain corpus, and will not scrape or use copyrighted text without permission; the final report will list the exact source and license of any corpus used beyond the repository's existing sample text. Second, because the model is trained on a narrow corpus at small scale, its outputs will be presented honestly as a demonstration of mechanism rather than as factually reliable text — the proposal explicitly avoids any claim that the system is suitable for unsupervised public-facing use. Third, the project will document known limitations (repetition, factual unreliability, sensitivity to prompt phrasing) alongside its results, rather than only showcasing favorable generations, in keeping with good academic practice around reporting machine learning results.

Chapter 4

RESEARCH PLAN AND BUDGET

4.1 Test Plan

Testing is planned before implementation is complete, rather than left until the end, because each feature in Section 3.3.1 is small enough to verify independently as soon as it exists. The table below lists the core test cases that will gate integration of each component.

Test Case	Condition	Expected Result
Tokenizer test	Input text is encoded and decoded; special tokens are handled.	Token ids decode back to the original string or an acceptable normalized form.
Dataset shape test	DataLoader returns input and target tensors for a batch.	Tensor shapes match (batch_size, context_length) with no off-by-one errors in the sliding window.
Attention mask test	A causal mask is applied before the softmax in self-attention.	No token's attention weights reference any future position.
Model forward test	A batch of token ids is passed through GPT_124.	Output shape is (batch, sequence_length, vocab_size).
Loss test	Cross-entropy loss is computed from logits and targets.	Loss is finite and decreases to near zero on a tiny overfit test set.
Generation test	A prompt is decoded for up to N tokens using top-k and temperature.	Output length and token validity respect the configured limits.
Checkpoint test	Model and optimizer state are saved and reloaded.	The reloaded model produces identical logits for identical input.

Test Case	Condition	Expected Result
Documentation/demo test	The README/demo steps are followed on a clean environment.	An evaluator unfamiliar with the code can reproduce the core demo without assistance.

Table 7: Testing Plan

4.2 Work Schedule and Gantt Chart

Tasks:

1. Requirement study and repository review
2. Literature review and dataset/tokenizer study
3. Dataset pipeline and data loader refactor
4. Model Implementation review and shape tests
5. Training loop, checkpointing, logging
6. Evaluation, decoding experiments, risk review
7. Unit testing and integration testing and
   Final report, presentation and demo preparation

Tasks	Weeks							
1	1							
2		2						
3			3					
4				4				
5					5 and 6			
6							7	
7								8

Figure 7: Project Gantt Chart (8-Week Plan)

Week	Planned Work
Week 1	Requirement study, repository review, and proposal finalization.
Week 2	Literature review and detailed study of the dataset/tokenizer pipeline
Week 3	Dataset pipeline refactor and data-loader cleanup.
Week 4	Model implementation review and tensor-shape testing.
Week 5–6	Training loop, validation loop, checkpointing, and metric logging.
Week 7	Evaluation, decoding experiments, risk review, and full test suite.
Week 8	Final report, presentation, demo preparation, and defence practice.

Table 8: Project Work Schedule (8 Week Plan)

4.3 Risk Analysis

Every project of this kind carries a small set of predictable risks. Figure 8 summarizes them on a likelihood/impact grid, and the table below lists the corresponding mitigation plan for each.

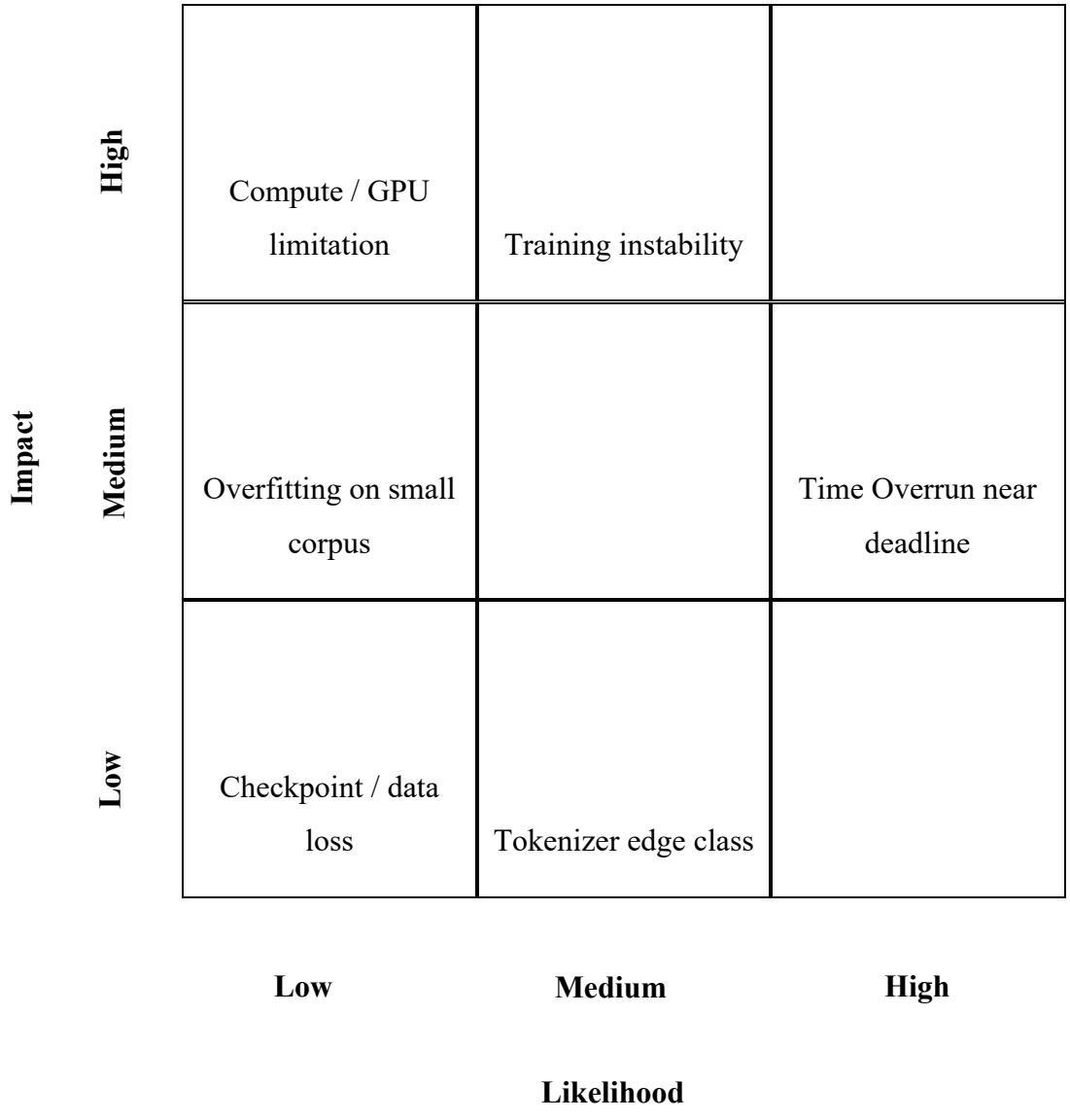

Figure 8: Risk Matrix

Risk	Likelihood	Impact	Mitigation
Limited or unstable GPU access	Medium	High	Use small batch sizes and gradient accumulation; fall back to free-tier cloud notebooks; keep a CPU-only smoke test path for grading.
Training instability (loss spikes, NaNs)	Medium	Medium	Apply gradient clipping, a learning-rate warmup schedule, and checkpoint frequently so training can resume from the last stable point.
Overfitting on a small corpus	Medium	Medium	Hold out a validation split, monitor validation loss separately from training loss, and apply dropout as already configured in the repository.
Tokenizer edge cases (unicode, punctuation)	Low	Low	Cover edge cases explicitly in the tokenizer unit tests; fall back to GPT-2's byte-level BPE, which has no out-of-vocabulary failure mode.
Time overrun near the final deadline	Medium	High	Follow the week-by-week schedule in Figure 7; keep documentation and diagrams updated throughout rather than left to the final week.
Checkpoint or experiment-log loss	Low	Medium	Save checkpoints and logs to version-controlled or cloud-backed storage at every evaluation interval.

Table 9: Risk Analysis and Mitigation

4.4 Budget

Estimated cost: negligible. the project requires no dedicated financial budget. All software (Python, PyTorch, and supporting libraries) is free and open-source, training runs on personally owned laptops or free-tier cloud GPU credits (e.g. Google Colab), and the corpus consists of small, public-domain or freely licensed text. The only marginal cost that could arise is optional paid cloud compute if a member chooses to

train beyond the free tier, which is a personal, not a project-level, expense and is not required to complete the work described in this proposal.

Chapter 5

EXPECTED OUTCOME

5.1 Impact and Significance

the immediate significance of this project is educational: it turns an abstract topic — how a large language model is trained — into a system the team has built, tested, and can explain line by line, which is a different and more durable kind of understanding than using a pretrained model through an API. Beyond the classroom, the finished repository, its test suite, and its documented evaluation protocol are reusable: they can serve as a teaching example for future cohorts, a base for the extensions described in Section 5.6, and evidence, at the team's own scale, of the convergence and scaling behaviour reported in the wider literature on transformer language models.

5.2 Model Training Results

Because training had not yet started at the time of writing, the curves below are illustrative rather than final: they show the shape of convergence the team expects to observe, based on the loss behaviour typically reported for small GPT-style models trained from scratch on a compact corpus (Karpathy, 2023). Both figures are native, editable charts backed by an embedded spreadsheet, so the placeholder series can be replaced directly with the team's actual training logs once available, without rebuilding the report.

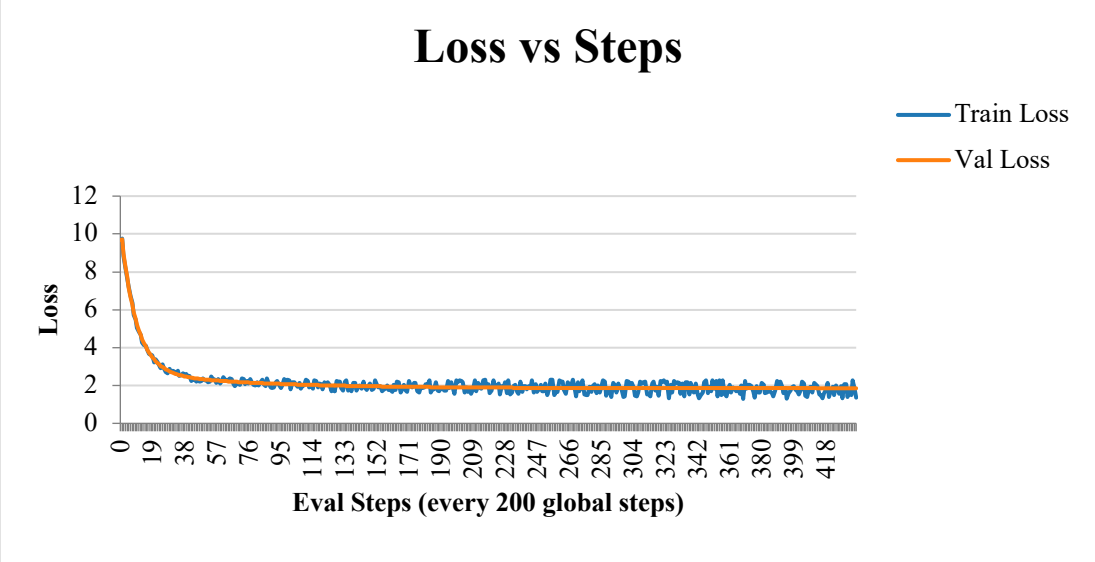

Figure 9: Loss vs Steps (Illustrative Training Curve)

Figure 9 illustrates the expected behaviour at fine granularity: a sharp initial drop within the first few evaluation steps as the model moves away from its randomly initialized weights, followed by a long, noisy plateau in which the training loss (blue) oscillates around a slowly falling validation loss (orange). The gap between the two curves is expected to stay small, since the corpus and model are both modest in size; a widening gap in the real logs would be the first signal of overfitting and would trigger the mitigation in Section 4.3 (Table: Risk Analysis and Mitigation).

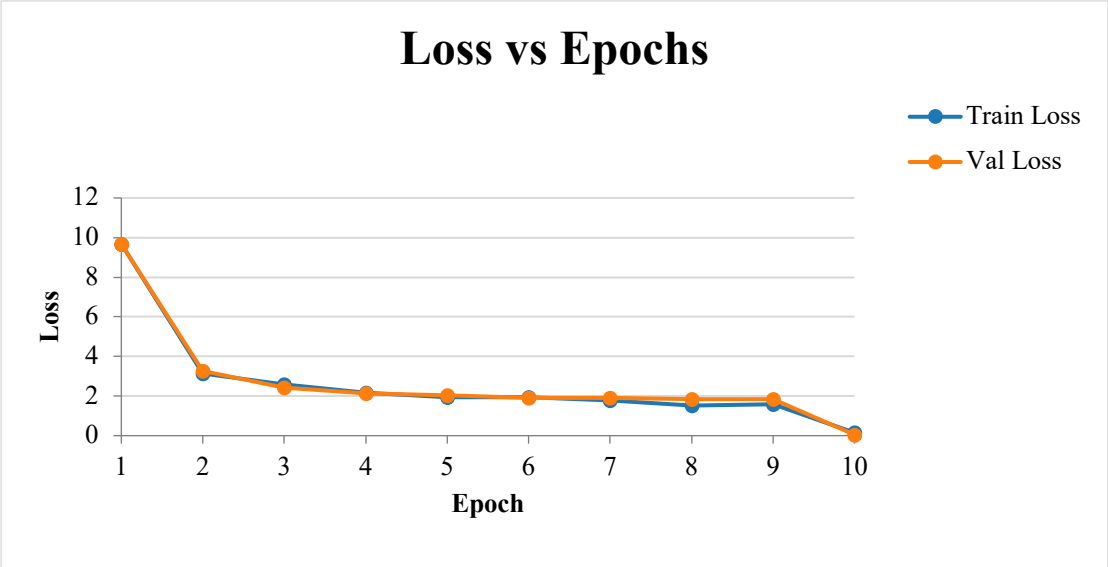

Figure 10: Loss vs Epochs (Illustrative Training Curve)

Figure 10 shows the same run summarized per epoch, which is the view used for at-a-glance reporting in the final defense. Most of the reduction in loss is expected within the first two to three epochs, after which further epochs yield smaller, steadier gains — the pattern the scaling-law literature (Kaplan et al., 2020) predicts for a fixed model size and dataset once the model has absorbed the easy, high-frequency patterns in the corpus.

5.3 Expected Deliverables

- A working, end-to-end GPT-style language model implemented in PyTorch, trainable on a chosen corpus.
- A documented, tested tokenizer and dataset pipeline built on GPT-2 byte-pair encoding.
- Training and validation logs that visibly demonstrate the model learning (decreasing loss, improving sample coherence).

- Generated text samples from a fixed set of prompts, produced under temperature and top-k sampling, with a discussion of their quality and limitations.
- A clear, evidence-based comparison of expected versus actual results, including an honest discussion of where the small-scale model falls short of production LLMs and why.
- A complete, reproducible repository with configuration logging, checkpointing, automated tests, and a one-command demo script.

5.4 Success Criteria

Success for this project is defined narrowly and honestly: the model must train without tensor-shape or masking errors, the causal mask must provably prevent any position from attending to future tokens, validation loss must be tracked and shown to decrease over training, and generated text must show visible, attributable learning from the training corpus — coherent words, plausible local grammar, corpus-specific vocabulary — even though it will not match the fluency of a production-scale LLM trained on internet-scale data.

5.5 Contributions

In summary, this proposal's contribution is not a new model architecture but a disciplined, fully verified engineering treatment of an existing one. Relative to the starting repository, the work adds: a modular code structure that separates the tokenizer, dataset, model, training, and generation concerns that were previously mixed together in notebook cells; an automated test suite that checks tensor shapes, causal masking, loss computation, checkpointing, and generation independently; a documented evaluation protocol with tracked loss and perplexity curves rather than ad-hoc inspection of sample output; a complete set of design artifacts (architecture, use case, ER, sequence, and class diagrams) describing a system that previously had none; and an explicit feasibility, risk, and ethical-use analysis that the original repository does not attempt.

5.6 Future Works

- Cross-lingual extension: repoint the same pipeline at a Nepali-language corpus to study how byte-pair tokenization and training dynamics behave on a morphologically different, comparatively low-resource language.

- Empirical scaling study: hold the corpus fixed and vary model depth and width to observe, first-hand, the loss-versus-parameter-count relationship described by Kaplan et al. (2020) and Hoffmann et al. (2022).
- Lightweight interface: add a minimal web front end for interactive prompting and side-by-side comparison of decoding strategies, built on top of the existing generation module without changing the model code.
- Instruction tuning: fine-tune the trained base model on a small instruction-following dataset to study, at small scale, how pretraining and instruction-tuning objectives interact.
- Reuse as a teaching artifact: package the finished repository, tests, and diagrams as a self-contained teaching example for future Research Fundamentals cohorts covering model evaluation, bias, or hallucination in generative systems.

Bibliography

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., & Clark, A. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*. https://arxiv.org/abs/2203.15556

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*. https://arxiv.org/abs/2001.08361

Karpathy, A. (2023). *nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs* [Software]. GitHub. https://github.com/karpathy/nanoGPT

beeks-code. (2026). *GPT-from-scratch: Building GPT-124M from scratch* [Software]. GitHub. https://github.com/beeks-code/GPT-From-scratch

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog, 1*(8). https://openai.com/research/language-unsupervised

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1715–1725. https://doi.org/10.18653/v1/P16-1162

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS), 30*, 5998–6008. https://arxiv.org/abs/1706.03762

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., & Funtowicz, M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 38–45. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Appendices

Appendix A: Items Added Beyond the Reference Materials

The following items were not present in the reference sample documents and have been added here to make this a stronger, more complete proposal:

- A dedicated risk analysis chapter with a visual risk matrix and a concrete mitigation plan for each identified risk (Section 4.3).
- An explicit ethical and responsible-use section addressing corpus licensing, honest reporting of limitations, and avoidance of public-facing deployment claims (Section 3.11).
- An expanded literature review with an explicit synthesis paragraph connecting the scaling-law literature to this project's scope decisions (Section 2.1).
- A scoped, written statement of what the project will not attempt, to set realistic evaluator expectations up front (Section 1.4).
- A feasibility analysis broken into technical, economic, and operational dimensions, matching standard engineering-proposal structure.
- All diagrams (architecture, use case, ER, two sequence diagrams, class diagram, Gantt chart, risk matrix) redrawn specifically for this proposal rather than left as placeholder figure captions.
- A clearly separated improvements list (Section 3.6) distinguishing what already exists in the public repository from what this project will add.

Appendix B: Scope Boundaries

This minor project does not claim to build a production chatbot, a commercial product, or a state-of-the-art language model. Its contribution is a transparent, fully tested, and fully documented implementation of the GPT mechanism at a scale appropriate for a single student team working within a single academic term, together with the engineering and design discipline needed to make that implementation reproducible and defensible.